Phizura: On-the-Fly Music Recording with Method Proxies

Domenico Cipriani[1], Sebastian Jordan Montaño[1] and Stéphane Ducasse[1]

[1]Univ. Lille, Inria, CNRS, Centrale Lille, UMR 9189 CRIStAL, Park Plaza, Parc scientifique de la Haute-Borne, 40 Av. Halley Bât A, 59650 Villeneuve-d'Ascq, France

Abstract

Live coding is a performance practice in which artists write and modify code in real time to generate music, visuals or poetry. Live coding turns coding into an artistic performance: an improvised practice in which the artist builds and reworks material on the fly, embracing the immediacy of creation in real time An open challenge for live coding is to capture the actual writing and evaluation of expressions rather than only the resulting audio, yielding a script that can be replayed yet remains modifiable and playable with other sounds This paper presents Phizura, a recording tool for on-the-fly music performances conducted with Coypu, a Pharo-based library and dialect for live music programming. To our knowledge, this is the first tool to record live coding performances via reflective instrumentation, replayable as self-contained scripts at evaluation granularity. Phizura leverages Method Proxies, a safe and fast message-passing control library, to non-intrusively intercept code evaluations and keystrokes during a performance. The recorded data, timestamped expressions and input events, can later be emitted as executable Pharo code that faithfully replays the performance with the original timing. We describe the architecture of Phizura, detail how Method Proxies enable a clean separation between recording logic and the Coypu codebase, and compare the approach to a previous Announcer-based implementation. Phizura provides clean, non-invasive instrumentation that captures both keystrokes and evaluations without polluting the original Coypu codebase.

Keywords

Pharo, Method interception, Code evaluation, Method Proxies, Coypu, Live-coding, On-the-fly music programming

1. Introduction

Live coding is an increasingly popular creative practice in which performers write and modify source code in real time to produce music, visuals, or other time-based media[1]. The field has developed a substantial academic presence: the International Conference on Live Coding (ICLC), now in its tenth year, has served as the primary venue for peer-reviewed research on the practice, attracting contributions from computer science, music, and the performing arts. Programming languages such as Tidal Cycles [**?**], Sonic Pi [**?**], and Orca [**?**] have popularized the practice, each offering alternative syntaxes and notations for expressing musical patterns. Within the Pharo ecosystem, Coypu[**?**] provides a library and a DSL for on-the-fly music programming, enabling performers to construct rhythmic sequences, melodic patterns, and to modify them interactively in a Playground.

To date, relatively little scholarly attention has been devoted to capturing and preserving live coding performances, particularly the generative process itself. Audio recording captures the sonic result but discards the compositional process: the sequence of code edits, evaluations, and their precise timing. Audio files are also large, difficult to share, and impossible to re-execute under a different setup. Compositional data, by contrast, enables exact replay of a performance with alternative sound synthesis, facilitates sharing between musicians as lightweight text files, supports analysis of live coding practice, and opens the door to automated post-processing of performances. This concern has been raised explicitly in the Tidal Cycles community[2], where users have discussed the need for tools that

IWST 2026: International Workshop on Smalltalk Technologies, July 4–10, 2026, Plovdiv, Bulgaria

mspgate@gmail.com (D. Cipriani); sebastian.jordan@inria.fr (S. Jordan Montaño); stephane.ducasse@inria.fr (S. Ducasse)

https://jordanmontt.fr (S. Jordan Montaño)

0009-0005-3023-3192 (D. Cipriani); 0000-0002-7726-8377 (S. Jordan Montaño); 0000-0001-6070-6599 (S. Ducasse)

[1]TOPLAP Manifesto at https://toplap.org/wiki/ManifestoDraft

[2]Tidal Club Forum, How to replay a live coding session. Link at https://club.tidalcycles.org/t/how-to-replay-a-live-coding-session/464, 2020.

store every evaluation with its timestamp and allow session-based replay. However, no integrated solution has emerged so far. With Phizura, recorded performances become living scripts that can be replayed, remixed, and rew-composed with new sounds.

This paper presents the design and implementation of Phizura, shows how Method Proxies[**?**] can be applied beyond profiling and debugging to support creative coding workflows, and compares the current approach to the previous Announcer-based implementation. The paper begins with background on Coypu and Method Proxies, then discusses the limitations of the earlier Announcer-based approach that motivated the redesign. It continues with the architecture and implementation of Phizura, the code emission and replay mechanism, and a discussion of related work. It closes with a reflection on generality, limitations, and future directions.

Future improvements includes a plan to expose a simpler public API that lets live coders define their own capture handlers, giving them more control over what is recorded and how it is processed. We also intend to support recording across multiple Playgrounds at once, to capture collaborative sessions.

2. Background

2.1. Live coding

Live coding (sometimes referred to as 'on-the-fly programming', 'just in time programming' and 'conversational programming') is a performing arts form and a creativity technique centred upon the writing of source code and the use of interactive programming in an improvised way[3]. This performance practice that takes shape as adventure and exploration [**?**]. It spans music, visual art, and poetry, and readily combines with dance and theater, making it a fertile ground for cross-disciplinary creation. The live coding practice resists fixed definitions and stays heterogeneous by nature, continually challenging its own self-understanding through the act of writing and rewriting code. A live coding musician works like an improvising composer, reshaping the entire structure of a piece by writing code on the fly. The code becomes the score, and the computer performs the music as the audience watches it being written. A substantial body of academic writing has grown around the practice, much of it gathered in the proceedings of the International Conference on Live Coding (ICLC), held since its first edition at the University of Leeds in 2015 [**?**] and convened roughly every year since across Europe, the Americas, and Asia. Many languages and environments, spanning a range of programming paradigms, have been built for live coding, including TidalCycles, Impromptu, Sonic Pi, Orca, and Mercury, among others[4]. Coypu is one of the most recent additions to this list, and the first to bring live coding into the Smalltalk world.

2.2. Coypu: programming music on-the-fly with Pharo

Coypu[**?**] is a library and domain-specific dialect for live coding music inside the Pharo environment. It acts as a client for an audio server, which can be either internal (a Phausto [**?**] DSP running within Pharo) or an external OSC-capable[5]application, typically SuperCollider via the SuperDirt engine, or a MIDI device. Coypu code is written and executed in a Pharo Playground; its vocabulary provides an extended set of methods for creating rhythmic and melodic sequences. Once these sequences are assigned to the Performance, an internal scheduler advances time and dispatches musical events to the selected audio server[6]. A musician performing with Coypu can also modify patterns in real time, muting or soloing individual voices, swapping instruments, and layering new material over what is already running. Every evaluation hits the running output immediately. The performance is shaped by what gets evaluated, when, and in what order and that is exactly what Phizura records.

[3]Quoting Wikipedia.

[4]Check https://github.com/toplap/awesome-livecoding#languages for a comprehensive list of live coding languages

[5]Open Sound Control (OSC) is a protocol for real-time communication between multimedia devices and software, typically transmitted over UDP, https://opensoundcontrol.stanford.edu.

[6]An experimental audio-visual live coding performance with Coypu, Phausto and Bloc at ESUG 2024 https://www.youtube.com/watch?v=8s3wypwqzSc&list=PLJ5nSnWzQXi-uyYOZIVgXg1up_bclQc7e&index=33&t=85s

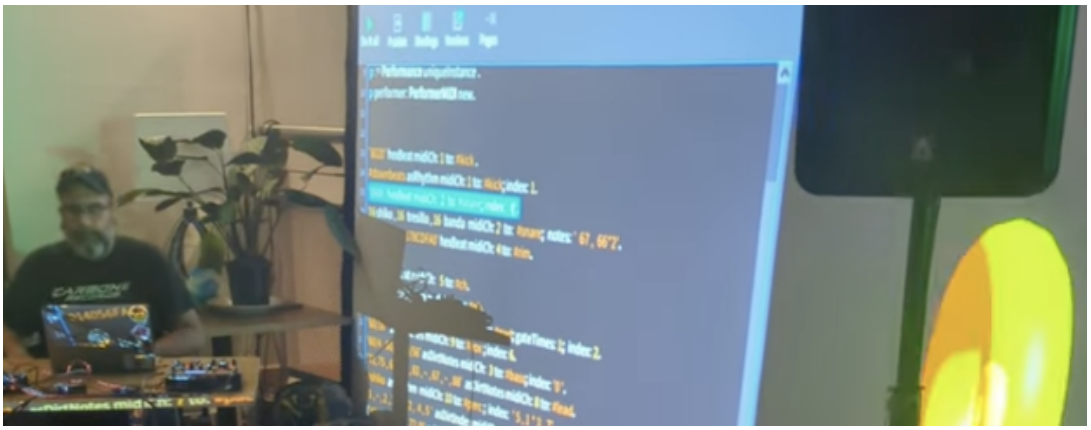

Figure 1: Live coding performance with Coypu and ChuGL at ADC26 Bristol social event.

2.3. Method Proxies: safe and fast method interception

MethodProxies [**?**] is a per-method instrumentation library for Pharo that implements *message-passing control*: an instrumentation paradigm in which user-defined code can be executed *before*, *after*, or *instead of* a method invocation, as well as during the stack unwind triggered by an exception or non-local return. **It leaves the source code of the instrumented application intact.** The library exposes a stratified architecture in which the user-facing API is reduced to subclassing a handler class (MpHandler) and overriding a small set of hooks; the framework manages the underlying instrumentation mechanics. The same handler can be associated with several proxied methods, which allows information to be aggregated across distinct call sites without duplicating logic.

Internally, each instrumented method is replaced in its method dictionary by a precompiled *trap method* that orchestrates the execution of the hooks. To avoid runtime compilation, MethodProxies ships a small set of precompiled trap prototypes. one per method arity. that are copied at installation time and literal patched Because the trap remains a regular CompiledMethod, it is JIT-compiled and optimized by the virtual machine as any other method. Stack unwinds are handled through the Pharo's built-in unwind mechanism rather than through the standard ensure: construct, avoiding the allocation of block closures and the associated stack reification cost.

MethodProxies also guarantees a number of properties that make it suitable for instrumenting arbitrary parts of the system, including the language kernel. It is *meta-safe* — instrumented methods can be invoked from within a handler without re-entering the instrumentation, since the active process is marked as meta-level during hook execution and the trap is bypassed while this flag is set. It is *unwind-safe*, supports dynamic installation and deinstallation at run time, and reports an average overhead of approximately 1.5 × with the meta-safety mechanism itself adding only about 9% [**?**].

2.4. Previous approaches to capturing performances

The first recording mechanism for Coypu was implemented by Redwane Engels during an internship and demonstrated at ESUG 2024 in Lille. It relied on Pharo's `Announcer` and `Announcement` classes, which implement the observer design pattern.

The approach was straightforward: the Coypu codebase itself was modified so that at each point where an evaluation could produce a musical effect, an announcement was explicitly fired. A subscriber object, the recorder, registered interest in these announcements and logged each one with a timestamp.

It functioned as intended, but introduced significant trade-offs. Recording logic was distributed throughout the codebase, and supporting each new recordable event required direct modifications to the source code. The recorder depended on specific announcement classes defined within the library, making the two systems difficult to evolve independently. Extending it to capture additional events, such

as keystrokes, cursor positions, or events from other tools, required yet more invasive changes. And every modification to the evaluation pipeline risked breaking or silently bypassing the recording hooks. These limitations motivated the redesign using Method Proxies, which externalizes the instrumentation entirely and it does not require modifying the source code.

3. MethodProxies-based vs. Announcer-based approaches

Table 1 summarizes the key differences between the two recording implementations. The most significant advantage of the Method Proxies approach is the complete elimination of coupling between the recording infrastructure and the domain library. In the Announcer-based design, every new recordable event required a two-step modification: first, defining a new `Announcement` subclass in Coypu, and second, inserting the corresponding announcement firing at the right point in the evaluation pipeline. Over time, this scattered recording logic throughout the codebase, making it increasingly difficult to distinguish Coypu's core musical functionality from the recording mechanism layered on top of it.

Table 1
Comparison of the Announcer-based and Method Proxies-based recording approaches.

Aspect	Announcer-based	Method Proxies-based
Coypu modifications	Required at each recordable site	None
Coupling	Tight (shared announcement types)	None (external instrumentation)
Adding new events	Modify Coypu + define announcement	Intercept target method
Removal	Must revert Coypu changes	Uninstall instrumentation
Keystroke capture	Not supported	Supported
Code emission	Not supported	Two modes (Performance, MoofLOD)

With Method Proxies, the instrumentation is installed externally on existing methods, and the handler receives the method's arguments reflectively. The original methods are entirely unaware of the instrumentation. Recording a new kind of event only requires intercepting the relevant method. No Coypu code needs to change. Removing the recording capability is equally straightforward: uninstalling the instrumentation restores the original methods, leaving no trace in the system.

In the next section, we will present how we used MethodProxies to implement Phizura.

4. Phizura: on-the-fly music recording

Phizura[7], pronounced `/fi'sura/` following Latin American Spanish phonetics, is a Coypu[8] replayer. Coypu is a library for live-generating electronic music. It is typically used by musicians and artists to perform live shows in which they generate and play music in real time. These performances are characterized by improvisation, which is the central artistic concept behind them. One of Coypu's core ideas is iconicity: the structure of the code should reflect the structure of the resulting music. This concept is based on the linguistic notion of form meaning similarity [**? ?**].

In some cases, artists and researchers aim to capture live coding performances [**?**] for later replay. Live coding involves not only writing code but performing it in real time, often in a musical concert setting. Phizura is designed to record all code executions exactly as written, along with all keystrokes. Timestamps are also captured to preserve the temporal sequence of the performance.

Thanks to Method Proxies, the implementation of Phizura was straightforward. We instrumented the methods responsible for code evaluation (compilation plus execution). Pharo already provides support for capturing keystrokes, so we integrated a call to Phizura within the existing callback.

The programmer can configure when to start and stop instrumentation. Because Method Proxies allows dynamic (de)instrumentation without restarting, the programmer record or stop recording at

[7]Phziura is available at GitHub at https://github.com/jordanmontt/Phizura
[8]Copyu can be found at github.com/lucretiomsp/Coypu

any time. Afterward, double dispatch is used to generate code that reproduces the session, preserving the original temporal flow by incorporating the recorded timestamps as delays. The generated code can also include custom actions, such as sending external signals to other devices for each code evaluation or keystroke.

The implementation is split across three packages.

- **Phizura** contains the recorder, the instrumentor, and the code emitter.
- **Phizura-Playground** provides a custom Playground that hosts a recording session and surfaces the recorder's controls in its toolbar.
- **Phizura-Tests** holds the test suite.

The rest of this section follows the three responsibilities the system actually carries out: instrumenting the Playground, collecting the events, and exposing the workflow to the performer.

4.1. Instrumentation

A single class, PhizuraEvaluationInstrumentor, decides which methods to intercept and installs a MpMethodProxy on each one. The two methods that are invoked during code evaluation methods are instrumented. Phizura covers all endpoints of code evaluation and it ensures that every evaluation is captured. Both proxies share the same handler instance.

The handler is the only piece of user-defined behavior the system needs. It is a subclass of MpHandler with a single method:

```
PhizuraHandler >> beforeExecutionWithReceiver:
    anObject arguments: anArrayOfArguments
    | argument |
    argument := anArrayOfArguments first.
    recorder addRecord:
        (PhizuraEvaluationRecordEntry new
            time: DateAndTime now;
            expression: argument;
            cursorPosition:
                playgroundPage text cursorPosition;
            yourself)
```

Listing 1: PhizuraHandler before-execution hook

The first argument of either intercepted method is the source string about to be evaluated. The handler timestamps it, reads the cursor position from the Playground page, and pushes the result onto the recorder. No Coypu class is modified, subclassed, or even mentioned: the instrumentation is entirely external, and uninstalling the proxies leaves the running image in its original state.

4.2. Recording

The recorder stores two kinds of events: a PhizuraEvaluationRecordEntry for each evaluated expression and a PhizuraKeyEventRecordEntry for each keystroke. Evaluations are recorded through Method Proxies described above. Keystrokes are captured independently from Method Proxies. A single recorder holds the entries in a chronological timeline.

PhizuraRecorder ties everything together. It owns the list of entries, holds a reference to the instrumentor, and toggles a recording flag. Starting a session creates the instrumentor and installs its proxies; stopping a session uninstalls them and clears the flag.

4.3. The Phizura Playground

Phizura ships its own Playground because the standard one offers no natural target for Method Proxies: the selected text flows through several layers of Spec machinery without ever passing through a stable

method one could hook onto. A small custom subclass solves this problem. PhizuraPlaygroundPagePresenter»doEvaluateAndGoFor: receives the selected text directly as its argument, which is the shape the handler expects.

The custom Playground is otherwise a thin layer over the standard one, with four subclasses. PhizuraPlaygroundPresenter substitutes the page class and stops the recorder when the window is closed. PhizuraPlaygroundPagePresenter is the central piece: it defines the method that the Method Proxies intercept, initializes the recorder, registers the keystroke listener, plugs in a custom interaction model so that the page can act as its own evaluation context, and extends the toolbar with five commands (Figure 2): *Phizurear* toggles recording, *Clean Phizura* resets the recorder, *Inspect Phizura* opens an inspector on the current session, and *Open MoofLOD* and *Open Records* emit the two flavours of replay script discussed in the next section. MoofLOD (Music on-the-Fly with Levels of Detail) is an ongoing research project undertaken jointly by two Lille-based teams, Evref (Inria) and MINT (CRIStAL). It augments live coding musical performances by exposing to the audience additional information about the musical structure, namely notes, rhythms, and instrument assignments, together with a real-time reproduction of the computer keyboard intended to make the performer's technical virtuosity visible.[9]

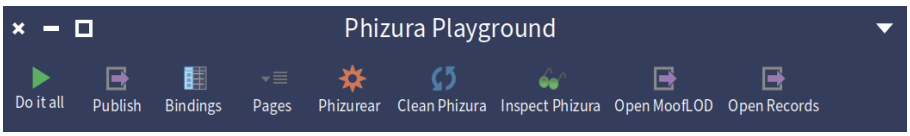

Figure 2: The Phizura Playground toolbar with the five extra commands.

A global variable, PhiRecorderInstance, holds the active recorder. It is initialized when the Phizura Playground opens, either from the Pharo world menu or via PhizuraPlayground open.

4.4. Code emission and replay

A session ends with a chronologically ordered list of timestamped entries in the recorder. Replay turns that list back into something the image can run. The job is done by PhizuraCodeEmitter, which walks the entries in order, subtracts each pair of consecutive timestamps to recover the original delays, and emits Pharo source separated by seconds wait statements. The whole script is wrapped in a forked block so that the waits do not freeze the UI process, and so that the performer can launch the replay from the same Playground that captured the session.

The emitter supports two modes, and the choice between them is made through a double dispatch with the record entries: the emitter asks each entry to render itself, and the entry calls back into the emitter with its own type information. New emission formats can therefore be added without touching the entry classes, and new entry types without touching the existing emitter methods.

The first mode, emitPerformanceCode, produces a runnable Coypu script. Keystroke entries are filtered out and only the evaluations remain, separated by the original delays. The result is self-contained: any image with the same Coypu setup can evaluate the script and reproduce the performance verbatim.

```
[
ep := EcoPhausto new.
1.0 seconds wait.
ep playFor: 128 bars.
1.5 seconds wait.
#(1 0 0 1 0 0 0 0) asSeq to: #kick.
1.8 seconds wait.
#rumba asRhythm to: #conga.
] fork.
```

[9]The central research question concerns whether, and to what extent, this additional information layer enhances audience engagement and comprehension.

Listing 2: Example of emitted performance code

The second mode, emitMoofLODCode, extends the performance code with OSC-compatible messages Each evaluated expression, keystroke, and cursor position is also signalled. The resulting script still reproduces the music, but it now also exposes everything that happened during the session to any OSC-capable consumer, opening the door to live visualizations, second-screen overlays, or synchronization with other tools.

```
[
'1' toLocal: 'keyPressed'.
0.269059 seconds wait.
'6' toLocal: 'keyPressed'.
0.06929 seconds wait.
'SPACE' toLocal: 'keyPressed'.
0.364536 seconds wait.
'P' toLocal: 'keyPressed'.
0.231215 seconds wait.
'L' toLocal: 'keyPressed'.
0.1328 seconds wait.
"... [keystrokes left out] ..."
'C' toLocal: 'keyPressed'.
0.284781 seconds wait.
'K' toLocal: 'keyPressed'.
0.200892 seconds wait.
'PERIOD' toLocal: 'keyPressed'.
0.164829 seconds wait.
16 plena to: #kick.
'16 plena to: #kick.' toLocal: 'evaluatedCommand'.
0.081839 seconds wait.
] fork.
```

Listing 3: Example of emitted MoofLOD code (middle elided)

Both modes are implemented on the record entries through emitPerformanceCodeOnEmitter:onStream: and emitMoofLODCodeOnEmitter:onStream:, completing the double dispatch with the emitter described above and keeping the iteration logic free of any type-specific code.

5. Related work

The challenge of recording and replaying live coding sessions has received relatively little attention in the literature. In the Tidal Cycles community, a forum discussion[10] explicitly outlines requirements that Phizura addresses: storing every evaluation with its timestamp, handling sessions with start and stop, and enabling replay. The thread concludes without a solution, noting the absence of such a tool across live coding environments.

The most directly comparable work is Live Writing by Lee and Essl [**?**], presented at ICLC 2015. During a Gibber live coding session in the browser, Live Writing records keystroke-level data: every character typed, cursor position, and timestamp. The system captures the typing performance, that is, how the code was physically entered. Live Writing is implemented via CodeMirror editor hooks in the browser, whereas Phizura uses reflective instrumentation through Method Proxies, requiring no modification to the editor or the live coding library.

[10]Tidal Club Forum, How to replay a live coding session, https://club.tidalcycles.org/t/how-to-replay-a-live-coding-session/464, 2020.

Another related effort is SHARP [**?**], a Pulsar[11] extension that visualizes the history of each instrument a live coder creates, providing an interactive tree-like structure to track how blocks of code evolve over time and to revisit previous states. It addresses the versioning aspect of the creative process. SHARP manages the evaluated expressions but does not capture timing or produce replayable scripts. Phizura and SHARP are complementary: SHARP preserves the structural history of code, while Phizura preserves the temporal performance.

6. Generality beyond music

This instrumentation targets all expressions evaluated in the Playground, not any Coypu-specific code. As a result, any expression evaluated in a Phizura Playground is captured[12], making the approach entirely generic. This opens several concrete uses beyond music. In the domain of visual live coding, performances using Bloc or Roassal for real-time graphics could be recorded and replayed with the same tool, without any modification to Phizura. Beyond live performance, Phizura can serve as a general-purpose prototyping recorder. Developers routinely explore APIs, test hypotheses, and prototype solutions through sequences of small evaluations. Phizura can capture these sessions as timestamped scripts, providing a lightweight form of session recording that preserves not just what was tried, but in what order and at what pace. Think documenting design decisions, reviewing your own process, or sharing a discovery workflow with other users.

A third application is pedagogical. An instructor demonstrating a concept in a Pharo Playground could record the session and distribute it as a small text file. Students would then replay the demonstration at their own pace, seeing each evaluation appear with the original timing. Unlike a video recording, the replayed script is live code: students can pause it, modify the expressions, and explore variations. Unlike a video recording, the replayed script is live code: students can pause it, modify the expressions, and explore variations. Unlike a static code listing, it preserves the temporal dimension, the order and rhythm of the instructor's thought process.

7. Conclusion and future work

This paper presented Phizura, a non-intrusive recorder for live coding performances in Pharo. Using Method Proxies, the instrumentation of the Playground's evaluation methods happens without modifying the Coypu codebase. The recorded timestamped expressions can be emitted as executable Pharo code that replays the performance with faithful timing. Compared to the previous Announcer-based implementation, the Method Proxies approach is modular, decoupled, and extensible.

Phizura demonstrates that Method Proxies, originally designed for profiling and debugging, are equally effective for creative applications. To the best of our knowledge, Phizura is the first tool to use reflective instrumentation for recording live coding performances, and the first to produce self-contained, evaluation-level replay scripts. The current implementation captures Playground evaluations but does not record workspace edits that are not evaluated. A performer may type, delete, and rearrange code extensively before pressing *Cmd+D*; only the final evaluated expression is recorded. The current implementation captures Playground evaluations but not mouse movements or actions. A performer can select, delete, cut, and paste code entirely through the mouse, with no keyboard interaction. Capturing these gestures would require additional instrumentation.

Finally, the current design supports a single recording session at a time, managed through a global variable. Collaborative live coding scenarios, where multiple performers evaluate code in separate Playgrounds, would require extending the architecture to support multiple concurrent recorders. One possible approach would be a dedicated Phizura instance per Playground, replacing the current Singleton. Future work will focus on two directions. First, we plan to expose a simpler public API within the

[11] Pulsar (https://pulsar-edit.dev) is a community-maintained, open-source text editor built on Electron, forked from GitHub's Atom after its discontinuation.

[12] Whether the code produces sound, generates visuals, manipulates data, or does anything else entirely.

Phizura package, allowing users to define custom handlers triggered by specific methods in their own libraries, giving live coders finer control over what is captured and how it is processed. Second, we aim to support recording across multiple Playgrounds simultaneously, enabling collaborative live coding sessions to be captured as a single, synchronized script.